Reinforcement Learning for Optimal Setpoint Determination in Energy Utility Systems

Master Thesis

Master of Science in Artificial Intelligence

submitted by

Ashish Thapa

5008270

born on 04 March 1999 in Kaski, Nepal

AI-Lab
Chair of Applied Mathematics
Brandenburg University of Technology Cottbus-Senftenberg

Reviewer:	Title and name of first reviewer
Reviewer:	Title and name of second reviewer
Adviser:	Title and name of adviser (optional)

Submission date: 25 June 2026

Declaration of independence

I hereby declare that I have written this thesis independently, without unauthorised help and exclusively using the sources indicated. The ideas taken directly or indirectly from external sources (including electronic sources) are labelled as such without exception. The sources used verbatim and in terms of content have been cited in accordance with the recognised rules of scientific work. This work has not yet been submitted for any other review or published elsewhere.

I agree that the thesis will be checked for plagiarism with the help of a plagiarism detection service.

Cottbus, 25 June 2026

Signature of the author

Task description

Energy utility systems are the backbone of modern energy infrastructure. Their complexity ranges from small, localized systems to vast, highly interconnected infrastructure networks. Most of them operate under dynamic conditions where multiple interacting variables influence operational efficiency, cost, reliability, and stability. Conventionally, optimal setpoints for such systems are determined using mathematical optimization. These approaches rely on accurate system models and are typically formulated as constrained optimization problems. While effective for well-defined and moderately sized systems, classical optimization becomes computationally expensive as system complexity increases. Furthermore, time-varying dynamics, uncertainty from the integration of renewable sources, and model-plant mismatches makes it challenging to use purely model-based approaches.

This thesis investigates whether reinforcement learning can find optimal operating setpoints for two case studies of different complexity: a battery energy storage system and a non-linear cookie production factory. For each system, the control task is formulated as a Markov decision process (MDP) with explicit state, action, and reward definitions aligned with operational objectives. The RL agent is trained to produce adaptive setpoint policies that minimize operating cost while satisfying process constraints and preserving stable operation. The learned policies are then benchmarked against established mathematical optimization methods to determine where RL provides practical benefits and where its limitations remain.

The evaluation of the proposed method will be conducted through systematic simulation studies on two energy system case studies of varying complexity. Performance will be assessed using quantitative metrics including total operational cost, constraint violation frequency, and computational time. The results will provide a structured comparison between reinforcement learning and classical optimization and will determine under which conditions RL offers practical advantages for real-time setpoint optimization in complex energy utility systems.

Abstract

Example summary in English.

Zusammenfassung

Beispielzusammenfassung in Deutsch.

Contents

1 Introduction **1**

2 Theory and Background **3**
2.1 Setpoint Optimization in Energy Utility Systems 3
2.2 Mathematical Optimization 3
2.3 Reinforcement Learning 5
2.3.1 TD3 and SAC 7

3 Use Cases **8**
3.1 Battery Energy Storage System 8
3.2 Cookie Production Factory 9

4 Methodology **11**
4.1 Problem Formulation 11
4.2 Reinforcement Learning Framework 12
4.3 Training Setup 13
4.4 Evaluation Metrics 15

5 Results and Discussion **16**
5.1 Battery Energy Storage System 16
5.2 Cookie Production Factory 17
5.3 Key Findings 17

6 Conclusion and Future Work **17**

1 Introduction

The operation of modern energy utility systems is shaped by dynamic conditions and multiple sources of uncertainty. Electricity prices vary over time, renewable generation is weather dependent, and storage systems couple current decisions with future operating flexibility [1]. At the same time, industrial energy systems often contain nonlinear component behavior, operating and coupling constraints that must be respected at every time step. The resulting setpoint selection problem is therefore not only an economic problem, but also a constrained sequential decision-making problem.

In many engineering applications, such setpoints are determined using mathematical optimization. This approach is attractive because physical constraints and objective functions can be formulated explicitly, and mature solvers can compute high-quality solutions for well-defined models. For energy systems, nonlinear programming methods such as IPOPT [2] are commonly used when component models, storage dynamics, and operational constraints are nonlinear [3]. In a real-plant application, Kyriakidis, Kansara, and Roldán Serrano show that deterministic and stochastic optimization methods can have high computational costs and slow convergence for large-scale nonlinear problems with complex constraints. They further show that forecast uncertainty, optimization-horizon length, and the available computation time affect operational decisions.[4].

Reinforcement learning (RL) provides a different perspective. Unlike classical control and optimization methods that solve a trajectory-optimization problem from scratch at each operating horizon, RL agents learn a policy or action-value function gradually through interactive experience. Once trained, this learned representation enables rapid selection of actions from the current observation [5]. This makes RL attractive for real-time or repeated decision-making problems, especially when the system is complex, nonlinear, and operates in a changing environment [6]. Nevertheless, RL does not automatically guarantee feasibility or optimality. For engineering use, a learned policy must be evaluated not only by its reward, but also by its operating cost, constraint satisfaction, and computation time relative to established optimization methods.

This thesis investigates whether RL can be used for optimal setpoint determination in energy utility systems. Two case studies are considered. The first is a battery energy storage system (BESS), which represents a comparatively low-dimensional storage-control problem with time-varying electricity prices. The second is a nonlinear cookie production factory with an electrified steam-generation utility system, wind-power availability, thermal energy storage, and coupled process constraints based on the benchmark systems described by Walden et al. and Kyriakidis et al. [3, 7].

The central research question of this thesis is:

> *Can reinforcement learning determine feasible and cost-effective operating setpoints for energy utility systems of varying complexity?*

This question is addressed through the following subquestions:

- How can the setpoint optimization problem be formulated as a Markov decision process with suitable states, actions, and rewards?
- Can deep reinforcement-learning policies learn continuous setpoint decisions that respect the system dynamics and operational constraints of the two systems?

- How does the learned policy compare with mathematical optimization in terms of operating cost, constraint satisfaction, and computation time?
- Under which conditions does RL offer a practical advantage, and where do classical optimization methods remain superior?

For each case study, the setpoint optimization problem is first formulated as a Markov decision process (MDP). Deep neural networks are used to parameterize the RL policies that map system observations to continuous setpoint decisions. The resulting policies are evaluated through deterministic rollouts in reinforcement-learning environments, and the resulting schedules are compared with mathematical optimization baselines. The comparison is based on total grid-electricity cost, constraint violations, and computation time. This separation is important because a policy with high cumulative reward is only useful if it remains feasible and economically meaningful under the original engineering objective.

The remainder of this thesis is structured as follows. Section 2 establishes the theoretical foundations of setpoint optimization and reinforcement learning, and introduces mathematical optimization as the benchmark used to evaluate the RL policies. It also describes the deep RL methods considered in this work. Section 3introduces the two case studies: a battery energy storage system and a nonlinear cookie production factory. Section 4 presents the methodological framework, covering the MDP formulation, environment design, policy-training procedure, optimization baselines, and evaluation metrics. Section 5 reports and discusses the experimental results, with particular emphasis on the comparison between RL policies and mathematical optimization. Finally, Section 6 summarizes the main findings, identifies the limitations of the study, and outlines directions for future work.

2 Theory and Background

This section provides the theoretical basis for the setpoint optimization problem studied in this thesis. The central question is how operating setpoints in energy utility systems can be selected over time so that costs are reduced while physical and operational limits are respected. The same problem can be viewed from two perspectives: as a model-based constrained optimization problem, and as a sequential decision-making problem that can be addressed with reinforcement learning. The section first introduces the fundamental concepts of RL and the MDP formulation [5], then presents deep RL methods for continuous control and the TD3 [8] and SAC [9] algorithms evaluated in this work.

Here, RL is not treated as a substitute for physical constraints. Instead, the setpoint-optimization problem is formulated as an MDP, with feasibility enforced where possible through action bounds, state-dependent mappings, and system dynamics, and otherwise discouraged through reward penalties. This distinction is important when comparing RL with mathematical optimization, because a low-cost policy is only useful if its decisions remain feasible.

2.1 Setpoint Optimization in Energy Utility Systems

Energy utility systems are operated by choosing setpoints for controllable components such as storage units, heat pumps, valves, and bypasses. A setpoint describes the desired value of a control variable. In the systems considered here, this may be the charging or discharging power of a battery, the bypass variable β_1, or a mass flow rate. These choices are not just technical details; they directly affect operating cost, energy efficiency, constraint satisfaction, and overall system reliability and lifetime [10].

Setpoint determination is also inherently sequential. The best operating point at one time step cannot be chosen independently of what happened before or what may be needed later. Storage systems make this especially clear. Charging a battery or a thermal energy storage can increase future flexibility, but it may also increase the current electricity cost. Discharging can reduce current grid consumption, but it leaves less stored energy for later. Good setpoint decisions must therefore account for inter-temporal dependencies and physical operating limits [3]. Each action affects both the immediate operating cost and the feasible decisions available at later time steps; consequently, its quality must be evaluated over the full planning horizon rather than solely at the current time step [11].

In practical operation, the controller also has to react to external signals such as electricity prices, renewable generation, production demand, and ambient conditions. Since these signals change over time, fixed rule-based strategies can become inefficient, and the operation can be formulated as a finite-horizon decision problem with time-varying boundary conditions [12, 6]. This motivates two formulations of the same setpoint determination task: constrained finite-horizon optimization and reinforcement learning.

2.2 Mathematical Optimization

Mathematical optimization provides a natural way to describe setpoint selection. In general, optimization means selecting the best decision from a set of feasible alternatives according to a defined objective. For energy systems, the objective is often to minimize operating cost or energy consumption, while the feasible set

is determined by physical equations and operational limits [3, 4]. A constrained optimization problem can be written as [13]:

$$\min_{x\in\Omega} f(x) \quad \text{s.t.} \quad h(x) = 0,\ g(x) \le 0, \tag{1}$$

Here, $x \in \Omega \subseteq \mathbb{R}^n$ denotes the decision variables, $f : \mathbb{R}^n \to \mathbb{R}$ is the objective function, $h : \mathbb{R}^n \to \mathbb{R}^p$ represents equality constraints, and $g : \mathbb{R}^n \to \mathbb{R}^q$ represents inequality constraints. The solution $x^\star$ is optimal if it minimizes the objective while satisfying all constraints.

For finite-horizon setpoint optimization, the decision vector contains the control actions for several time steps. The objective is usually the accumulated operating cost over the horizon. The constraints describe system dynamics, energy balances, storage bounds, and terminal requirements. This time-coupled structure is important because a state such as battery state of charge or thermal storage temperature depends on previous decisions.

The difficulty of an optimization problem depends strongly on its mathematical structure. Linear and mixed-integer linear formulations are widely used in energy-system applications because simplified models can often be solved reliably. However, realistic energy conversion systems frequently include nonlinear component behavior, continuous control variables, and coupled dynamic constraints. Such systems lead to nonlinear programming (NLP) formulations, which are generally harder to solve and may contain local optima [13]. As the number of time steps, decision variables, nonlinear component models, and coupling constraints increases, the optimization problem generally becomes larger and more difficult to solve within a short operational time window [7].

Another important distinction is whether the problem is deterministic or stochastic. In deterministic optimization, all relevant parameters, such as electricity prices, wind availability, and demand, are assumed to be known over the optimization horizon. Stochastic optimization represents uncertain quantities through random variables or scenarios. This is closer to real operation, but it also increases model size and computational complexity.

Nonlinear constrained problems are usually solved with dedicated numerical solvers. Interior-point methods are common in this setting. Wächter and Biegler introduce IPOPT as an open-source solver for large-scale nonlinear programming that uses a primal-dual interior-point method with a filter line-search strategy [2].

To illustrate the interior-point principle, consider the general problem in Eq. (1). Introducing slack variables converts the inequality constraints into $g(x) + s = 0$, where $s \ge 0$. A logarithmic barrier with parameter $\mu > 0$ restricts the iterates to the interior, $s > 0$, and leads to the subproblem

$$\begin{aligned} \min_{x,s} \quad & f(x) - \mu \sum_{j=1}^{q} \ln(s_j) \\ \text{s.t.} \quad & h(x) = 0, \qquad g(x) + s = 0, \qquad s > 0. \end{aligned} \tag{2}$$

The logarithmic barrier prevents the iterates from reaching the constraint boundary. IPOPT solves a sequence of such subproblems using a primal-dual Newton method. With equality multipliers y and inequality multipliers z, the perturbed Karush–Kuhn–Tucker conditions are

$$\begin{aligned} \nabla f(x) + J_h(x)^\top y + J_g(x)^\top z = 0, \\ h(x) = 0, \qquad g(x) + s = 0, \\ Sz = \mu \mathbf{1}, \qquad s > 0,\ z > 0, \end{aligned} \tag{3}$$

where J_h and J_g are the Jacobians of the equality and inequality constraints, respectively, and $S = \mathrm{diag}(s)$. As μ is reduced towards zero, the perturbed complementarity condition $Sz = \mu\mathbf{1}$ approaches the KKT complementarity condition of the original constrained problem [2].

Nocedal and Wright note that the performance of nonlinear optimization algorithms depends on formulation quality, particularly the constraint structure, variable scaling, and initial point [13].

These considerations motivate the investigation of reinforcement learning as a complementary approach to model-based optimization. The following section introduces its fundamental principles and its formulation for setpoint optimization.

2.3 Reinforcement Learning

Reinforcement learning (RL) offers a different view of the same setpoint problem. Instead of solving an optimization problem from scratch at every decision time, an agent learns from repeated interaction with an environment. At each time step, the agent observes the current state, selects an action, receives a reward, and observes the next state. Over time, it adapts its policy so that actions leading to high long-term reward become more likely [5].

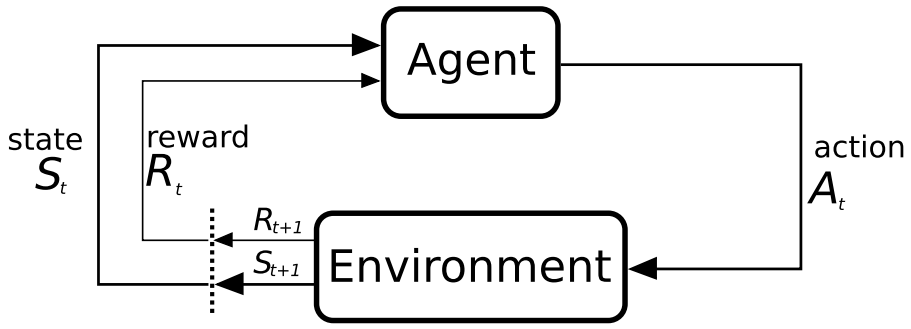


Figure 1: Interaction loop between the reinforcement-learning agent and the environment in a Markov decision process [14].

Formally, RL problems are commonly modeled as a Markov Decision Process (MDP), defined by the tuple [5]

$$(\mathcal{X}, \mathcal{A}, p, r, \gamma_{\mathrm{RL}}),$$

where

- $\mathcal{X}$ is the set of states representing the system,
- $\mathcal{A}$ is the set of actions,
- $p(x' \mid x, a)$ defines the transition dynamics,
- $r(x, a)$ is the reward function,
- $\gamma_{\mathrm{RL}} \in [0, 1]$ is the discount factor.

Under the Markov property, the current state contains all information required to predict the next state, given the selected action. A policy $\pi(a \mid x)$ describes how the agent selects actions. To evaluate a decision, RL considers both its immediate reward and the rewards obtained thereafter. For a finite horizon of N decision steps, this quantity, called the return, is

$$G_t = \sum_{i=0}^{N-t-1} \gamma_{\mathrm{RL}}^i r(x_{t+i}, a_{t+i}),$$

and the learning objective is to find a policy that maximizes the expected return. The value function and action-value function of a policy π are defined as

$$v_t^\pi(x) = \mathbb{E}_\pi[G_t \mid X_t = x], \qquad q_t^\pi(x,a) = \mathbb{E}_\pi[G_t \mid X_t = x, A_t = a].$$

The value function estimates the expected return from a state, whereas the action-value function estimates the expected return from a state-action pair. These estimates allow the agent to consider the future consequences of a decision, not only its immediate reward.

For a finite-horizon problem, the optimal action-value function can be written recursively as

$$q_t^\star(x,a) = r(x,a) + \gamma_{\mathrm{RL}}\, \mathbb{E}\big[v_{t+1}^\star(x') \mid x, a\big].$$

Here, x' denotes the next state. Rather than calculating this expectation from a known system model, RL learns an approximation of the action-value function from sampled transitions (x_t, a_t, r_t, x_{t+1}). The policy is then improved to select actions with higher estimated long-term return. This illustrates the central idea of RL: actions are evaluated not only by their immediate reward, but also by their expected future consequences [5].

Classical dynamic programming can solve MDPs through backward recursion. However, this approach becomes impractical when the state or action space grows, a difficulty known as the curse of dimensionality [5]. Realistic energy systems often have continuous states, continuous actions, and long horizons, so exact dynamic-programming solutions are often computationally infeasible.

RL addresses this limitation by learning approximate policies from interaction data. A trained policy can then compute an action directly from the current observation. In model-free RL, the learning algorithm does not require an explicit analytical transition model for planning, although training still requires interaction data. In engineering applications, this data is usually generated with a simulator or digital model, so the quality of the learned policy depends strongly on the quality of the training environment.

To locate the algorithm used in this thesis, it is helpful to briefly classify the main families of RL algorithms. RL methods are not separated by one single list of types; instead, they are usually described through several overlapping distinctions:

- Model-based and model-free methods: model-based methods learn or use a model of the environment dynamics for planning. Model-free methods learn directly from sampled transitions.

- Value-based and policy-based methods: value-based methods learn value or action-value functions and choose actions from them. Policy-based methods directly optimize a parameterized policy.

- Actor-critic methods: actor-critic methods combine ideas above. The actor represents the policy, and the critic estimates values or action-values used to improve that policy.

- On-policy and off-policy methods: on-policy methods learn from data generated by the current policy. Off-policy methods can also reuse data generated by previous policies or exploratory behavior.

- Tabular and deep RL methods: tabular methods store values explicitly for small state and action spaces. Deep RL uses neural networks as function approximators and is therefore more suitable for large or continuous spaces.

The relevant setpoints in this thesis, including battery power, temperature levels, mass flow rates, and bypass positions, are continuous quantities. Discretizing them would either exclude useful operating points or create a very large action space. TD3 and SAC are therefore selected because they learn parameterized policies that output continuous actions directly.

Both methods use an actor-critic architecture. The actor maps the current state to continuous setpoint decisions, while the critic estimates their expected long-term effect on operating cost and future system states. TD3 and SAC are applied to learn continuous setpoint policies for both case studies.

2.3.1 TD3 and SAC

Twin Delayed Deep Deterministic Policy Gradient (TD3). TD3 uses a deterministic actor, $\mu_\theta(x)$, that maps the current state directly to a continuous action. Two critics estimate the action value, and the smaller estimate is used when constructing the learning target to reduce overestimation bias. TD3 further stabilizes learning through delayed actor updates and target-policy smoothing, in which small noise is added to target actions during critic updates [8].

$$J_{\mathrm{TD3}}(\theta) = \mathbb{E}_{x\sim\mathcal{D}}\left[q_{\phi_1}\big(x, \mu_\theta(x)\big)\right].$$

Here, $J_{\mathrm{TD3}}(\theta)$ is the actor objective; θ denotes the actor parameters; $\mathcal{D}$ is the replay buffer from which states x are sampled; $\mu_\theta(x)$ is the deterministic action proposed by the actor; and q_{ϕ_1} is the first critic, parameterized by ϕ_1, which estimates the action value. The TD3 actor is trained to maximize this objective, producing one deterministic action, $a = \mu_\theta(x)$, for each state.

Soft Actor-Critic (SAC). SAC instead uses a stochastic actor, $\pi_\theta(a \mid x)$, that defines a distribution over continuous actions. In addition to maximizing expected return, SAC includes an entropy term that encourages exploration while training. Like TD3, SAC uses two critics and experience replay; however, its stochastic policy and entropy objective distinguish it from the deterministic TD3 policy [9].

$$J_{\mathrm{SAC}}(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{N-1} \gamma_{\mathrm{RL}}^t \left(r(x_t, a_t) + \alpha\mathcal{H}(\pi(\cdot \mid x_t))\right)\right].$$

where $J_{\mathrm{SAC}}(\pi)$ is the objective of the stochastic policy π; $\mathbb{E}_\pi$ denotes the expectation over trajectories generated by that policy; N is the number of decision steps; γ_{RL} is the discount factor; $r(x_t, a_t)$ is the reward at time t; α weights the entropy term; and $\mathcal{H}(\pi(\cdot \mid x_t))$ is the policy entropy.

For a state x, the policy entropy is defined as

$$\mathcal{H}\big(\pi(\cdot \mid x)\big) = -\mathbb{E}_{a\sim\pi(\cdot|x)}\left[\log \pi(a \mid x)\right].$$

where a is an action sampled from the policy, and $\log \pi(a \mid x)$ is the logarithm of its probability density. During training, entropy is estimated from sampled actions as $\mathcal{H}(\pi(\cdot \mid x)) \approx -\log \pi(a \mid x)$ [9].

The two algorithms therefore provide a comparison between deterministic and stochastic actor–critic policies for continuous setpoint optimization. Their performance is evaluated using operating cost, constraint satisfaction, and computation time.

3 Use Cases

Describes the two systems considered in this thesis and their operational challenges.

3.1 Battery Energy Storage System

Describes the battery system used as the first case study.

Battery energy storage systems (BESS) are electrochemical devices that store electrical energy for later use and provide grid services such as energy arbitrage, peak shaving, and short-term balancing. A typical BESS is characterized by its usable energy capacity, maximum charge/discharge power, round-trip efficiency, and state-of-charge (SOC) operating limits. Real-world operation must balance economic objectives (e.g., buying low / using stored energy when prices are high) with engineering constraints (SOC bounds, power limits, efficiency losses).

BESS presents several challenges. Decisions are inherently time-coupled because current charging/discharging changes future availability. In addition, uncertain exogenous signals (market prices) require policies that are robust to forecast errors, and hard feasibility constraints must be enforced at every time step. These properties make BESS an illustrative case study for comparing optimization-based and reinforcement-learning approaches to real-time setpoint determination in energy systems.

A battery energy storage system provides flexible, time-shiftable power by charging when electricity prices are low and discharging when prices are high to reduce grid procurement cost. In this thesis, the battery is modeled using the `BatteryEnv` environment: the agent's observation includes the state-of-charge (SOC), a normalized current price, a normalized time index, and the normalized price forecast for the complete scheduling horizon; the action is a continuous charge/discharge command $a \in [-1, 1]$ scaled to grid-side power $P_t^{\mathrm{S}} \in [-P_{\max}, P_{\max}]$.

The SOC dynamics are modeled as

$$\mathrm{SOC}_{t+1} = \mathrm{SOC}_t + \frac{P_{\mathrm{actual},t}\,\Delta t}{E_{\max}},$$

where $P_{\mathrm{actual},t}$ is the effective storage-side power after efficiency losses, $E_{\max}$ the usable energy capacity, and Δt the time step. The implemented environment uses the sign convention $P_t^{\mathrm{S}} > 0$ for charging and $P_t^{\mathrm{S}} < 0$ for discharging, with a simplified symmetric efficiency coefficient η. The superscript S identifies power exchanged with the electricity supply grid. The system is subject to the following operational constraints:

$$0 \leq \mathrm{SOC}_t \leq 1, \quad \forall t, \tag{4}$$

$$-P_{\max} \leq P_t^{\mathrm{S}} \leq P_{\max}, \quad \forall t, \tag{5}$$

$$P_{\mathrm{actual},t} = \eta P_t^{\mathrm{S}}, \quad \forall t. \tag{6}$$

where $P_{\max}$ is the maximum charge/discharge power. Constraint (4) ensures the battery operates within safe operating levels, Constraint (5) enforces physical power limitations, and Constraint (6) models the efficiency losses used in the RL environment. The environment enforces feasibility by dynamically limiting the commanded power at each step so that the SOC remains within bounds.

The goal is to reduce the cost of electricity from the grid, which is calculated as the price multiplied by the power used. The environment gives the negative of this cost as the reward. In this work, there is no on-site generation and no selling back to the grid. The battery charges from the grid when prices are low and discharges to serve local demand when prices are high. Main challenges include respecting SOC and power limits, accounting for efficiency losses, handling uncertain future prices, and managing the fact that actions now affect future energy availability. These features make the BESS a suitable test case for reinforcement learning, where a policy must balance using energy now against saving it for likely higher-price periods while always obeying feasibility limits.

3.2 Cookie Production Factory

Introduces the industrial production system considered as the second case study.

The second use case represents a non-linear cookie production factory with an electrified steam-generation utility system. The production process requires a constant supply of process heat in the form of superheated steam, while the utility system must decide how to operate the energy conversion components under fluctuating wind generation and electricity prices. The operational objective is to satisfy the steam demand while minimizing grid electricity cost over a finite scheduling horizon. The system model is based on the industrial power-to-heat system introduced by Walden et al. [3] and later used as a renewable steam-generation benchmark by Kyriakidis et al. [7].

The system consists of four main units: a wind turbine (WT), a closed reverse Brayton-cycle high-temperature heat pump (HTHP), a sensible thermal energy storage (TES), and a steam generator (SG). The WT supplies renewable electricity, while additional electricity can be purchased from the grid. The HTHP uses a waste-heat air stream as heat source and delivers high-temperature process heat to an intermediate thermal-oil loop. The thermal oil acts as heat transfer fluid and can be routed through the TES or directly to the SG by controllable bypasses. The SG provides the required steam conditions for the consumer factory. The HTHP and SG are represented by polynomial surrogate models, while the TES is modeled as a simplified single energy reservoir, and the charging/discharging behavior is represented using efficiency equations.

The corresponding finite-horizon optimization problem is formulated over a dis-

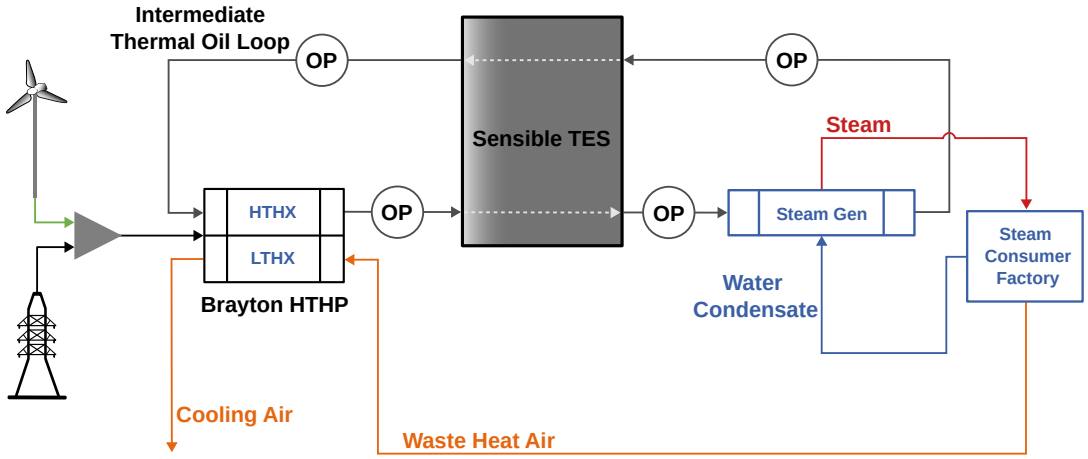


Figure 2: Schematic representation of the electrified steam-generation system serving the cookie production factory. Reproduced from Kyriakidis et al. [7]; the system was originally introduced by Walden et al. [3].

crete time horizon with time index $t \in \{1, \ldots, n\}$:

$$\min_{\mathbf{x}} \quad J = \sum_{t=1}^{n} P_t^{\mathrm{S}} S_t \Delta t, \tag{7}$$

$$\text{s.t.} \quad P_t^{\mathrm{S}} + P_t^{\mathrm{W}} = 3F_{\mathrm{HTHP}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{8}$$

$$T_{\mathrm{II}}^t = F_{\mathrm{HTHX}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{9}$$

$$T_{\mathrm{IV}}^t = F_{\mathrm{LTHX}}\left(T_{\mathrm{I}}^t, \dot{m}_{\mathrm{I}}^t, T_{\mathrm{III}}^t, R^t\right), \tag{10}$$

$$T_2^t = \beta_1^t T_{\mathrm{II}}^t + \left(1 - \beta_1^t\right) T_1^t, \tag{11}$$

$$T_{\mathrm{I}}^t = \beta_2^t T_3^t + \left(1 - \beta_2^t\right) T_4^t, \tag{12}$$

$$T_2^t = 201.92 + \frac{1819.32}{3\dot{m}_{\mathrm{II}}^t}, \tag{13}$$

$$T_3^t = 196.3 - \frac{188.4}{3\dot{m}_{\mathrm{I}}^t}, \tag{14}$$

$$\dot{Q}_{s,\mathrm{ch}}^t = 3\dot{m}_{\mathrm{II}}^t c_{p,\mathrm{f}}\left(T_{\mathrm{II}}^t - T_1^t\right)\left(1 - \beta_1^t\right), \tag{15}$$

$$\dot{Q}_{s,\mathrm{dch}}^t = 3\dot{m}_{\mathrm{I}}^t c_{p,\mathrm{f}}\left(T_4^t - T_3^t\right)\left(1 - \beta_2^t\right), \tag{16}$$

$$\dot{Q}_{s,\mathrm{ch}}^t \dot{Q}_{s,\mathrm{dch}}^t \leq \gamma, \tag{17}$$

$$T_1^t = T_{\mathrm{II}}^t - \epsilon_{\mathrm{ch}}\left(T_{\mathrm{II}}^t - T_s^{t-1}\right), \tag{18}$$

$$T_4^t = T_3^t - \epsilon_{\mathrm{dch}}\left(T_3^t - T_s^{t-1}\right), \tag{19}$$

$$T_s^t = T_s^{t-1} + \frac{\dot{Q}_{s,\mathrm{ch}}^t - \dot{Q}_{s,\mathrm{dch}}^t}{m_s c_{p,\mathrm{s}}} \Delta t, \tag{20}$$

$$T_s^0 = \tilde{T}_0, \qquad T_s^n = T_s^0, \tag{21}$$

$$T_{\mathrm{I}}^t \in [177, 250], \qquad \dot{m}_{\mathrm{I}}^t \in [5, 16], \tag{22}$$

$$T_{\mathrm{III}}^t \in [60, 100], \qquad R^t \in [0.8, 1.53], \qquad \beta_1^t, \beta_2^t \in [0, 1]. \tag{23}$$

Here, S_t denotes the spot electricity price, P_t^{S} denotes grid power, and P_t^{W} denotes available wind power at time step t. The objective in Eq. (7) minimizes the electricity cost of grid power consumption. The power balance in Eq. (8) couples grid electricity, wind power, and HTHP demand. The remaining constraints describe the HTHP and SG surrogate models, bypass mixing, TES charging and discharging, the prohibition

of simultaneous charge and discharge, cyclic storage operation, and the admissible operating ranges of the main decision variables.

The main control decisions are the bypass variables β_1 and β_2, the compressor-speed-related variable R, the mass flow rate $\dot{m}_{\mathrm{I}}$, and the HTHP inlet temperature T_{I}. The bypass variables determine the operating mode of the system. In charging mode, heat from the HTHP is routed into the TES; in discharging mode, stored heat is released to support the steam-generation process; and in idle or bypass operation, the storage is not actively charged or discharged. Simultaneous charging and discharging is prohibited through a complementarity constraint.

In the reinforcement-learning environment, the agent observes the normalized TES temperature, current spot electricity price S_t, available wind power P_t^{W}, the previous operating mode, and the corresponding look-ahead windows $S_{t+1:t+H}$ and $P^{\mathrm{W}}_{t+1:t+H}$. The action is a continuous vector containing β_1, β_2, R, $\dot{m}_{\mathrm{I}}$, and T_{I}. The reward is based on the negative grid-electricity cost, with additional penalties for violating operational requirements such as TES temperature bounds, invalid mode behavior, coupling residuals, and deviation from the cyclic terminal storage condition.

This use case is more complex than the battery system because it combines nonlinear surrogate equations, thermal storage dynamics, multiple coupled continuous control variables, and hard process constraints. It therefore provides a suitable second benchmark for evaluating whether reinforcement learning can learn feasible and cost-effective setpoint policies for industrial energy utility systems, and for comparing the learned policy against the existing mathematical optimization formulation.

4 Methodology

Describes how the optimization problem is formulated as a reinforcement learning problem.

The previous section introduced the two energy utility systems considered in this thesis and described their operational objectives and constraints. This section explains how these system-specific optimization problems are reformulated as reinforcement-learning problems. In particular, the setpoint optimization task is expressed as a finite-horizon sequential decision problem, where an agent observes the current physical system state together with current and forecast exogenous variables, such as electricity prices and available wind power, selects continuous control actions, and receives a reward based on operating cost and constraint satisfaction.

4.1 Problem Formulation

Formulates the system control problem as a reinforcement learning problem.

For both use cases, the objective is to determine operating setpoints over a discrete time horizon such that the total operating cost is minimized while all relevant constraints are respected. Let $t \in \{0, \dots, N-1\}$ denote the discrete time index, S_t the spot electricity price, P_t^{S} the grid power, and P_t^{W} the available wind power. At each time step, the system is described by a state vector x_t, the controller selects an action vector a_t, and the system evolves to the next state x_{t+1} according to the underlying system dynamics and the exogenous inputs S_t and, for the cookie-factory use case, P_t^{W}.

The resulting decision problem is formulated as a Markov decision process (MDP)

$$\mathcal{M} = (\mathcal{X}, \mathcal{A}, p, r, \gamma_{\mathrm{RL}}),$$

where $\mathcal{X}$ is the state space, $\mathcal{A}$ is the action space, $p(x_{t+1} \mid x_t, a_t)$ denotes the transition dynamics, $r(x_t, a_t)$ is the reward function, and γ_{RL} is the discount factor. The agent aims to learn a policy $\pi(a_t \mid x_t)$ that maximizes the expected cumulative reward [5]:

$$\max_{\pi} \; \mathbb{E}_{\pi}\left[\sum_{t=0}^{N-1} \gamma_{\mathrm{RL}}^{t} r(x_t, a_t)\right].$$

Since the original engineering objective is cost minimization, the reward is defined as the negative operating cost, augmented by penalty terms for constraint violations where required. In this way, feasible and economically favorable operation corresponds to a high cumulative reward. The following subsections describe how this general formulation is implemented in the reinforcement-learning framework, training setup, and evaluation procedure.

4.2 Reinforcement Learning Framework

Explains the structure of the RL agent and learning algorithm.

The reinforcement-learning framework used in this thesis follows the same general structure for both use cases. The environments are implemented using Gymnasium, while the continuous-control policies are trained using Stable-Baselines3. At time step t, the environment supplies the agent with a forecast-augmented state x_t, comprising the current physical system state, current exogenous information, a normalized time indicator, and the forecast profiles for the complete scheduling horizon. The agent selects a normalized continuous action a_t, which the environment maps to physical control setpoints. The environment then advances the system dynamics and returns the next state x_{t+1}, the reward r_t, and the Gymnasium termination signals. A transition is therefore generated as

$$\tau_t = (x_t, a_t, r_t, x_{t+1}, d_t),$$

where d_t indicates whether the episode has ended. In the present environments, reaching the end of the fixed scheduling horizon is represented by the `terminated` signal. The transitions are stored in a replay buffer and sampled repeatedly during training, enabling the off-policy learning used by SAC. SAC augments the conventional return-maximization objective with an entropy term during training to encourage exploration of the continuous action space. This implementation follows the general agent–environment interaction illustrated in Figure 1, extended by forecast-augmented states, environment-specific action mappings, and constraint handling.

During training, an action sampled from the stochastic actor is first processed by an environment-specific action map

$$a_t^{\mathrm{phys}} = \mathcal{R}(x_t, a_t),$$

where $\mathcal{R}$ converts the normalized policy output into physical setpoints and applies environment-specific feasibility handling. Hard constraints are enforced through action bounds, state-dependent mappings, or the system dynamics, while requirements that are not enforced directly are represented by penalty terms in the reward.

For the battery system, the state supplied to the agent consists of the current state of charge, the normalized current spot electricity price S_t, the normalized position within the scheduling horizon, and the normalized 24-step price forecast $S_{0:H-1}$. The complete forecast is supplied unchanged at every decision step, so no end-of-episode padding is required. The continuous scalar action $u_t \in [-0.8, 1]$ represents the battery charging or discharging command: positive values correspond to charging and negative values to discharging. A smooth, state-dependent action map converts this normalized command into grid-side battery power while progressively restricting charging and discharging near the upper and lower SOC limits. The resulting grid-side power determines the electricity cost, whereas the efficiency-adjusted battery-side power determines the SOC transition.

The reward has the same general form in both environments:

$$r_t = -C_t - \sum_{\ell} \lambda_\ell v_{\ell,t} - \lambda_{\mathrm{T}} v_{\mathrm{T},t},$$

where C_t is the grid-electricity cost at time step t, $v_{\ell,t}$ are constraint-violation terms, and λ_ℓ are penalty weights. The terminal term $v_{\mathrm{T},t}$ is only active at the end of an episode. In the battery case it penalizes an insufficient terminal SOC, while in the cookie factory it penalizes deviation from the cyclic terminal TES temperature. This reward design converts the original cost-minimization problem into a return-maximization problem without removing the need for physical feasibility.

More specifically, the battery reward implemented in the environment is

$$r_t = -\frac{S_t P_t^{\mathrm{S}} \Delta t}{1000} - \mathbf{1}_{\{t=N-1\}} \lambda_{\mathrm{SOC}} \max\left(SOC_{\min}^{\mathrm{terminal}} - SOC_N, 0\right),$$

where S_t is the spot electricity price in EUR/MWh, P_t^{S} is the grid-side battery power in kW, and Δt is the step duration in hours. The factor 1000 converts the energy charge to euros. The terminal term is applied only at the end of an episode and penalizes a final SOC below the prescribed minimum. Intermediate SOC feasibility is handled directly by the battery dynamics and state-dependent power mapping rather than by a constraint-violation penalty.

After training, the stochastic exploration behavior is no longer required for comparing schedules. The learned policy is therefore evaluated deterministically by selecting the mean action predicted by the actor at each time step. This produces one reproducible setpoint trajectory for a given price and wind profile and allows direct comparison with the mathematical optimization baseline in terms of cost, constraint satisfaction, and computation time.

4.3 Training Setup

Describes the training environment, reward function, and hyperparameter configuration.

Training is performed in reinforcement-learning environments before the learned policies are evaluated on fixed test trajectories. This is necessary because direct trial-and-error interaction with real energy utility systems would be unsafe and costly. The environment provides the transition dynamics, cost calculation, constraint handling, and terminal condition for each episode. The learning algorithm only receives observations, rewards, and termination signals through the environment interface.

For both use cases, one episode corresponds to a 24-step scheduling horizon. During training, the start index of the episode is sampled randomly from the available

time-series data. This random-window strategy exposes the agent to different price and renewable-generation patterns instead of repeatedly starting each episode at the same time of day. Both environments use the same grid-price series; the cookie factory environment additionally samples the corresponding wind-availability values for the same 24-hour window. The forecast horizon is also set to 24 steps, so every observation contains the current operating state together with the complete episode profiles of the relevant exogenous signals.

The training environments are wrapped with monitoring and vectorized environment interfaces. Monitoring records episode rewards and lengths for later inspection, while the vectorized wrapper provides the interface expected by the Stable-Baselines3 implementation of SAC. Only one environment instance is used at a time in the current setup, so the vectorization is mainly an interface layer rather than a parallel simulation strategy.

The BESS reward is the negative grid-side electricity cost at each time step, with an additional terminal penalty if the final SOC falls below the desired terminal level. The cookie factory reward also starts from the negative grid-electricity cost, but includes additional penalties for different constraint violations. The penalty terms are therefore more important in the second use case, because feasibility cannot be represented by simple action bounds alone.

The main SAC hyperparameters used in the implementation are summarized in Table 1. The same learning-rate schedule and discount factor are used for both use cases to provide a consistent baseline configuration for continuous-control learning. Differences between the two training setups mainly arise from the different state variables, action spaces, and constraint structures of the two environments.

Parameter	Battery system	Cookie factory
Algorithm	SAC	SAC
Policy	MLP policy	MLP policy
Replay buffer size	200000	200000
Batch size	256	256
Learning rate	linear schedule from $3 \cdot 10^{-4}$	linear schedule from $3 \cdot 10^{-4}$
Discount factor γ_{RL}	0.9999	0.99
Target smoothing coefficient τ	0.005	0.005
Training frequency	1 step	1 step
Gradient steps per update	1	1
Episode length	24 steps	24 steps
Forecast horizon	24 steps	24 steps

Table 1: Main SAC training settings used for the two reinforcement-learning environments.

For the both cases we use the Stable-Baselines3 SAC default architecture, consisting of two hidden layers of 256 neurons with ReLU activation functions for actor and critic networks. In both cases, trained models are saved after learning and are later loaded for deterministic evaluation. This separation between training and evaluation is important: stochastic actions are useful during learning, but deterministic actions are used for the final comparison with mathematical optimization.

4.4 Evaluation Metrics

The trained RL policies are evaluated through deterministic rollouts on fixed 24-hour evaluation windows. Deterministic evaluation means that the actor returns one action for each observation without sampling exploration noise. This makes the resulting setpoint trajectory reproducible and allows a direct comparison with the mathematical optimization baseline on the same grid-price and wind-availability profiles.

The primary economic metric is the total grid-electricity cost over the evaluation horizon:

$$J_{\text{grid}} = \sum_{t=0}^{N-1} C_t,$$

where C_t is the grid-electricity cost at time step t. For the battery system, this cost is computed from the electricity price and the battery power exchanged with the grid. For the cookie factory, it is computed from the grid power required by the heat-pump system after accounting for available wind power. Since the engineering objective is cost minimization, a lower value of J_{grid} indicates better economic performance.

The cumulative reward is reported as a secondary metric:

$$R_{\text{ep}} = \sum_{t=0}^{N-1} r_t.$$

This value is useful for analyzing the RL objective because it combines operating cost with penalty terms. However, it is not used as the only comparison criterion, since the reward may include artificial penalty weights that are not part of the physical operating cost. Therefore, total cost and constraint satisfaction are evaluated separately.

Constraint satisfaction is measured by aggregating the violation terms recorded during the rollout. For a set of constraint residuals or violation magnitudes $v_{\ell,t}$, the total violation is defined as

$$V_{\text{tot}} = \sum_{t=0}^{N-1} \sum_{\ell} v_{\ell,t}.$$

In addition, the number or frequency of time steps with nonzero violations is considered:

$$f_{\text{viol}} = \frac{1}{N} \sum_{t=0}^{N-1} \mathbb{I} \left(\sum_{\ell} v_{\ell,t} > \varepsilon \right),$$

where ε is a small numerical tolerance. For the battery system, the relevant checks include SOC bounds, power limits, and the terminal SOC requirement. For the cookie factory, the violation categories include bypass feasibility, complementarity of charging and discharging, TES temperature bounds, grid-power bounds, coupling residuals, and the terminal cyclic TES condition.

The evaluation also records trajectory-level operating indicators. For the BESS, these include the SOC trajectory, commanded and effective battery power, terminal SOC, and cumulative cost.

Finally, computation time is used to assess practical applicability. For mathematical optimization, the relevant value is the solver time required to compute a complete schedule for the evaluation horizon. For RL, the one-time training effort is reported separately from the online evaluation time. After training, the relevant online cost is the time required for a policy forward pass and one environment step at

each decision time. This distinction is important because RL can require substantial training effort, but a trained policy can usually produce setpoints much faster than solving a nonlinear optimization problem from scratch for every horizon.

5 Results and Discussion

Presents and analyzes the experimental results.

5.1 Battery Energy Storage System

The battery case study was first used to compare different learning-rate schedules for SAC. Each configuration was trained and evaluated over 32 runs on the same battery environment and evaluation setup. Table 2 summarizes the mean cumulative reward and the corresponding total grid-side objective. Lower objective values are better, while higher rewards are better.

Learning-rate schedule	Cumulative reward	Total objective [EUR]
Constant	8.9891 ± 0.0325	-8.9891 ± 0.0325
Linear	8.9962 ± 0.0286	-8.9962 ± 0.0286
Power-law	8.9809 ± 0.0439	-8.9809 ± 0.0439

Table 2: Battery SAC performance for different learning-rate schedules over 32 runs.

The linear schedule achieved the best mean reward and the lowest mean objective value among the tested configurations. However, the differences between the three schedules are small compared with the observed run-to-run variation. This indicates that the battery environment is not highly sensitive to the exact learning-rate decay shape in this experiment. The linear schedule was therefore used as a stable baseline for the subsequent comparison against mathematical optimization.

The trained SAC policy was then compared with the Pyomo optimization model over 1000 randomly sampled 24-hour price windows. Both methods used the same price windows and the same grid-side objective. Table 3 summarizes the resulting objective values, computation times, and SOC ranges.

Metric	SAC policy	Pyomo optimization
Mean objective [EUR]	-7.9557 ± 0.8811	-8.2391 ± 0.8410
Objective range [EUR]	$[-10.3298,\ -5.2038]$	$[-10.5381,\ -5.6214]$
Mean computation time [s]	0.0180 ± 0.0086	0.3041 ± 0.1859
Time range [s]	$[0.0104,\ 0.1305]$	$[0.0779,\ 1.8481]$
Mean maximum SOC	0.7767 ± 0.1533	0.8001 ± 0.1543
Mean minimum SOC	0.2116 ± 0.1524	0.1968 ± 0.1493

Table 3: Comparison between the trained SAC policy and the Pyomo battery optimization model over 1000 random 24-hour windows.

The Pyomo model achieved a slightly lower mean objective value than the trained SAC policy. The mean signed objective gap was 0.2834 ± 0.1536 EUR, and the mean absolute gap was 0.3040 ± 0.1072 EUR. This corresponds to a mean relative absolute gap of $3.76 \pm 1.46\%$. Since the objective is minimized, this shows that the

RL policy remained slightly less cost-optimal than the mathematical optimizer, but the economic gap was small in the current comparison.

The SOC trajectories are also similar in the current comparison. The SAC policy reached a mean minimum SOC of 0.2116 and a mean maximum SOC of 0.7767, while the Pyomo solution reached a mean minimum SOC of 0.1968 and a mean maximum SOC of 0.8001. This indicates that both methods used a comparable portion of the available storage flexibility, with the optimization baseline still finding slightly better charge and discharge timing on average.

The main advantage of the SAC policy was computation time. The trained policy required on average 0.0180 s for a 24-hour rollout, while the Pyomo model required 0.3041 s on average to solve the corresponding optimization problem. This corresponds to an average speedup of approximately 16.86×. The result therefore illustrates the central trade-off observed in the battery case: RL provides very fast online setpoint generation after training and approaches the mathematical optimization result closely, while the direct optimizer still provides the best objective value.

5.2 Cookie Production Factory

5.3 Key Findings

6 Conclusion and Future Work

Summarizes the main findings and outlines possible future work.

References

[1] M. Khalid. "Smart grids and renewable energy systems: Perspectives and grid integration challenges". In: *Energy Strategy Reviews* 51 (2024), p. 101299. DOI: 10.1016/j.esr.2024.101299.

[2] A. W"achter and L. T. Biegler. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical Programming* 106 (2006), pp. 25–57. DOI: 10.1007/s10107-004-0559-y.

[3] J. V. Walden et al. "Nonlinear operational optimization of an industrial power-to-heat system with a high temperature heat pump, a thermal energy storage and wind energy". In: *Applied Energy* 344 (2023), p. 121247. DOI: 10.1016/j.apenergy.2023.121247.

[4] L. Kyriakidis, R. Kansara, and M. I. Roldán Serrano. "Energy management of industrial energy systems via rolling horizon and hybrid optimization: A real-plant application in Germany". In: *Energies* 18.15 (2025), p. 3977. DOI: 10.3390/en18153977.

[5] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[6] E. Pilling, M. Bähr, and R. Wunderlich. *Reinforcement Learning Methods for the Stochastic Optimal Control of an Industrial Power-to-Heat System*. Dec. 2025. DOI: 10.48550/arXiv.2411.02211. URL: http://arxiv.org/abs/2411.02211 (visited on 02/11/2026).

[7] L. Kyriakidis, M. A. Mendez, and M. B"ahr. "A hybrid algorithm based on Bayesian optimization and Interior Point OPTimizer for optimal operation of energy conversion systems". In: *Energy* 312 (2024), p. 133416. DOI: 10.1016/j.energy.2024.133416. URL: https://linkinghub.elsevier.com/retrieve/pii/S036054422403192X.

[8] S. Fujimoto, H. van Hoof, and D. Meger. "Addressing function approximation error in actor-critic methods". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1587–1596. URL: https://proceedings.mlr.press/v80/fujimoto18a.html.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. PMLR, 2018, pp. 1861–1870.

[10] A. P. Tran and F. C. Yücel. "Safe and efficient control of a Brayton cycle heat pump using reinforcement learning". In: *Proceedings of the 16th International Modelica&FMI Conference*. Vol. 218. Linköping University Electronic Press, Oct. 2025, pp. 1017–1026. DOI: 10.3384/ecp2181017. URL: https://ecp.ep.liu.se/index.php/modelica/article/view/1389.

[11] P. H. Takam and N. Fruiba. *Stochastic optimal control problems for the cost-optimal management of a standalone microgrid*. Oct. 2025. DOI: 10.48550/arXiv.2511.00167. URL: https://arxiv.org/abs/2511.00167.

[12] Q. Meng, S. Hussain, F. Luo, Z. Wang, and X. Jin. "An online reinforcement learning-based energy management strategy for microgrids with centralized control". In: *IEEE Transactions on Industry Applications* 61.1 (2025), pp. 1501–1510. DOI: 10.1109/TIA.2024.3430264.

[13] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. New York: Springer, 2006. DOI: 10.1007/978-0-387-40065-5.

[14] EBatlleP. *Markov diagram v2*. Licensed under CC BY-SA 4.0; reproduced without modification. Wikimedia Commons. Jan. 23, 2020. URL: https://commons.wikimedia.org/wiki/File:Markov_diagram_v2.svg (visited on 08/20/2026).